

ZTracker — Quick User Guide

Fiji / ImageJ plugin · a two-part pipeline: Z-Projection maker → 3D Z-Coordinate Extractor

What this plugin does

ZTracker turns your **2D cell tracks** into **3D tracks** by adding a Z (depth) coordinate to every detection. It works in two parts, each its own item under **Plugins > ZTracker**, run in order:

1. **Z-Projection + Origin Map** (Part 1) — from a raw Z-stack, it builds the depth-coded projection images and a `.json` depth map.
2. **3D Z-Coordinate Extractor** (Part 2) — it reads those, looks up the depth under each tracked point, and saves 3D tracks.

If a colleague or an earlier run already gave you the depth-coded images and the `.json` map, you can skip Part 1 and go straight to Part 2.

Two extractors, and they are not interchangeable. Which one you use depends on where your depth images came from:

- **Indexed images + a JSON map** (what Part 1 makes) → use **3D Z-Coordinate Extractor**. Each pixel is a *code* that looks up a real depth in the `.json` file. Parts 1 and 2 are one pipeline: Part 2 reads both of those, plus a **tracking CSV** that you supply.
- **Fiji TopoJ height maps, no JSON** → use **3D Z-Extractor (TopoJ / direct-Z)**, a separate entry point that is **not** part of that pipeline. A TopoJ pixel holds an encoded slice counter, **not** a depth in μm , so the tool will not run until you give it four numbers read off the **source stack you ran TopoJ on**. That is its extra **Step 2** — Part 2 (alternative), below, explains all four.

So the two do **not** share every step: the TopoJ extractor has one more, and each later step is numbered one higher. Getting the pairing right is your job — the tools check bit depth, not where a folder came from, so a folder that loads is not proof you picked the right tool.

Part 1 — Z-Projection + Origin Map

Makes the two inputs Part 2 needs, starting from a raw Z-stack. Open it via **Plugins > ZTracker > Z-Projection + Origin Map**.

What Part 1 needs

A **dataset folder** holding your raw Z-stack, in either of two layouts — you say which with the **Input type** dropdown.

Input type = **Z-layer sub-folders** (the default): sub-folders named by their depth in μm (e.g. `-300`, `-299`, ...), each holding one TIFF per time frame (same filename across depths):

```
dataset/
├─ -300/   frame_0001.tif, frame_0002.tif, ...
├─ -299/   frame_0001.tif, ...
└─ ...
```

Name each depth folder as the number alone — `-300`, not `z-300` or `-300um`. A decimal point is fine (`-300.5`), a decimal comma is not. A sub-folder whose name is not a plain number is **not read as a depth**: it is skipped with a note in the Log and the run carries on. That is deliberate, so `notes` or `QC` folders can sit beside your data — but it means a misnamed depth folder is quietly left out of the projection instead of stopping the run, and the depths you get back will be missing it.

*Input type = **TIFF stacks***: one multi-slice TIFF **per time frame**, sitting straight in the folder — the slices inside each file are the depths:

```
dataset/
├─ 00001.tif    ← time frame 1, one slice per depth
├─ 00002.tif    ← time frame 2
└─ ...
```

Any numbering works (`00001.tif`, `7.tif`, `img_0010.tif`) — files are ordered by the number they **end with**. With this layout the depths are read from each slice's label inside the file (the `z = -400.000` labels Fiji writes for a Z-stack), since there are no depth-named folders to read them from. If a slice's label has no depth in it, the tool stops and tells you which slice — better than quietly using a wrong depth.

Whichever layout you use:

- **Every time-frame file must carry a number in its name, and no two may resolve to the same number.** A file's time-frame number is the **last run of digits anywhere in its name**, with leading zeros ignored — so `7.tif` and `0007.tif` are both time frame 7 and clash, while `z_0008.tif` is separate. A name with no digits states no position in the sequence, and two files reading as the same number cannot both hold that position. Either is refused before anything is written, with every offending name listed, so you can rename them in one go.
- **Your raw slices can be 8-, 16-, or 32-bit grayscale.** Part 1 only compares pixel brightness between depths, so the bit depth affects precision, not whether it works — use whatever your microscope produced. But **all the depths of one time frame must be the same bit depth**: mixing 8- and 16-bit is rejected, because the brighter scale would win everywhere and the depths you got back would be wrong. Convert them all to the same type (**Image > Type**) and re-run.
- **Colour (RGB) images are rejected**, with a message naming the offending layer or slice — there is no single brightness to compare in a colour pixel. Convert to grayscale first (**Image > Type > 8-bit or 16-bit**) and re-run.
- **Every time frame should be the same pixel size.** One that differs from the first frame written is skipped rather than written out, because a folder of mixed-size images cannot be loaded by Part 2 at all. The run continues and warns at the end. The frame everything is compared against is simply the first one that projected successfully, and is never itself checked — so if the odd-sized frame happens

to come first, every conforming frame is skipped against it instead. The warning names that reference frame, so check it before re-cropping anything.

Two habits prevent most Part 1 problems.

- **Check the Input type matches the folder you picked.** The two layouts look nothing alike on disk, and choosing the wrong one is the commonest reason a run finds no data at all. The grey structure line under **Scope** in the dialog spells out what the tool expects to find.
- **Give every run its own empty output folder.** Re-using an output folder, or copying two runs' results together afterwards, mixes files that were never meant to share a folder — and Part 2 then either refuses it (two images claiming the same frame number) or is handed frames of different sizes. Part 1 keeps a single run over one dataset consistent; it cannot police a folder you assembled yourself.

Memory, before you start a big one. What Fiji needs depends on which **Input type** you picked — not on how many time frames you have, since those are handled one at a time either way.

- **Z-layer sub-folders** (the default) is the expensive layout: every depth of a time frame is held at once, roughly **width × height × 4 bytes × number of depths**. A 1051 × 1674 frame with 401 depths comes to over 2.6 GB.
- **TIFF stacks** are cheap: slices are read one at a time, so the cost stays modest — a few tens of MB — no matter how deep the stack is. The exception is a **compressed** TIFF, which has to be unpacked whole, so budget roughly the size of the file itself.

These are estimates from the formula rather than measured requirements — put your own frame size and depth count in. If a run stalls or fails on a large dataset, raise the ceiling at **Edit > Options > Memory & Threads** and restart Fiji. That setting is what Fiji is allowed to use, not how much RAM your machine has.

Running Part 1

In the dialog you pick, top to bottom:

- **Input type** (asked first) — *Z-layer sub-folders* or *TIFF stacks*, as above.
- **Scope** — *Single dataset* (one dataset folder) or *Batch* (a folder holding many such datasets). A grey line under it spells out exactly what your input folder should contain for the combination you picked. In **Batch**, a candidate folder whose depth folders are all misnamed is skipped with no message while the rest of the run reports success, so check the dataset count in the Log is the number you expected. In **Single** the same folder fails outright and names the sub-folders it rejected.
- **Input folder** and **Output folder**.
- **Projection** — *Max-Z* (keeps the **brightest** pixel at each position — the usual choice for fluorescence), *Min-Z* (keeps the **darkest**), or *Both* (runs both, into separate `max_z` and `min_z` folders).
- **Z-origin bit depth** — *16-bit* (the default; smaller and faster), *32-bit*, or *Both*. Leave it on **16-bit** unless a downstream tool specifically needs 32-bit — it's the faster choice and handles any realistic number of Z-layers.

The 8-bit "raw" preview picture is always saved — there's nothing to switch on.

What Part 1 produces

For each dataset, a `max_z_<dataset>` (or `min_z_<dataset>`) folder in your output folder; in Batch these are grouped one level down under a `max_z` / `min_z` folder. Inside:

Output	What it is
z_origin/ (16-bit) and/or z_origin_32bit/ (32-bit)	The depth-coded projection TIFFs — point Part 2's Z-origin TIFF folder here. Only the bit depth you chose is written (16-bit by default).
z_layer_mapping.json and/or z_layer_mapping_32bit.json	The pixel-code → depth map — this is Part 2's Z-mapping JSON . Only the bit depth you chose is written: 16-bit writes <code>z_layer_mapping.json</code> , 32-bit writes <code>z_layer_mapping_32bit.json</code> , Both writes both. They hold the same depths, but use the one whose name matches the TIFF set you point Part 2 at.
raw/	A plain 8-bit projection picture for looking at (Part 2 ignores it).

The 16-bit and 32-bit depth images hold the same information — use 16-bit unless you have more than ~65,000 Z-layers (essentially never), in which case use the 32-bit set.

Both input types produce **exactly the same output**, so Part 2 doesn't care which one you used.

The `z_origin` folder and its matching `.json` map are exactly what Part 2 reads next.

Part 2 — 3D Z-Coordinate Extractor

Takes your **2D cell tracks** (from TrackMate or another tracker) and adds the **Z (depth) coordinate** to every detection, turning 2D tracks into 3D tracks.

It reads depth from the **Z-origin projection TIFF images** (from Part 1), where each pixel's value is a code that looks up a real depth in micrometers from the `.json` file. For each tracked point, the plugin looks at the pixel(s) under it, converts them to depth, and saves the result.

What you need before you start

File	What it is
Z-mapping JSON	A <code>.json</code> file that maps pixel codes to depths in μm , e.g. <code>{"0": -600.0, "1": -599.0, ...}</code> . (Part 1 makes this.)
Z-origin TIFF folder	A folder of TIFF images (one per time frame), 16-bit or 32-bit. These are the depth-coded projection images. (Part 1 makes this.) 8-bit and colour (RGB) TIFFs are rejected with a clear message — a pixel here is a code that has to reach into the depth map, and 8-bit only counts to 256.

File	What it is
Tracking CSV	Your tracks exported from TrackMate (or another tracker). Must contain X, Y, Frame, and Track ID columns.

Four things the Z-origin TIFF folder must get right. All are checked as soon as the folder loads, so you'll see the message before the wizard goes any further:

- **Point it at the depth-coded folder, not the preview.** Part 1 writes `z_origin` (or `z_origin_32bit`) and a `raw` folder side by side. `raw` holds the plain 8-bit preview pictures rather than depth codes, so choosing it is rejected — pick the `z_origin` folder whose bit depth matches the `.json` map you are using.
- **Every filename must contain the frame number.** The frame index is the **last run of digits anywhere in the name**, so a prefix with its own numbers is fine — `z_origin_0007.tif` and `z_origin_32bit_0007.tif` both load as frame 7. Any zero-padding width works, and the widths may differ from file to file within the same folder — `z_7.tif`, `z_0008.tif`, and `z_00000009.tif` load as three separate frames (7, 8, 9), not as a clash. A file with **no digits at all** in its name (e.g. `frame.tif`) is rejected, and the message lists every offending file so you can rename them in one go. Note this rule is **looser than the TopoJ / direct-Z extractor's**, which needs the number at the very **end** of the name: `topoj_0007_final.tif` loads fine here as frame 7, but that same file is rejected by the TopoJ extractor. (It doesn't work the other way round — anything the TopoJ extractor accepts is fine here too, with the same frame number.)
- **No two filenames may resolve to the same frame number.** `run1_0007.tif` and `run2_0007.tif` both read as frame 7, so one would replace the other and vanish from the stack. The folder is refused, naming the frame number and every file that landed on it. This almost always means two runs were mixed into one folder — keep each run's output separate.
- **Every image must be the same size as the first one.** If any TIFF's width or height differs from the first frame's, the folder is rejected and the message names the file along with both sizes. This applies to images that are **larger** as well as smaller — a larger frame used to load quietly, cropped to the first frame's top-left corner, and now stops the run instead. If you're deliberately working with a cropped subset, crop every frame to the same size first.

Opening the plugin

In Fiji, go to **Plugins > ZTracker > 3D Z-Coordinate Extractor**. The plugin runs as a simple 6-step wizard. You can move, resize, and read the Fiji **Log** window at any time while a step is open — it stays usable.

The 6 steps

1 Pick your 3 input files

Use the `...` browse buttons to select the **Z-mapping JSON**, the **Z-origin TIFF folder**, and the **tracking CSV**. Click OK.

2 Confirm the CSV format

The plugin asks how to read your CSV (header row, rows to skip, default cell radius). TrackMate exports have 3 extra info rows under the header — the defaults usually handle this. If you're unsure, leave the defaults and continue. If your CSV has a radius column, any cell that is blank, zero, negative or unreadable falls back to the default radius — the Log reports how many rows that happened to and names the first few.

3 Confirm the columns

The plugin auto-detects which columns are X, Y, Frame, and Track ID and shows its guess. Check they look right and correct any that are wrong, then continue.

4 Line up the frames (offset)

Trackers often number frames starting at 0, while TIFF files often start at 1. This step lets you shift the CSV frames so they match the TIFF files. A suggested offset is already filled in (**+1 is the most common correct value**).

As you change the number, the box shows a live **per-track verdict**, and a full table is printed to the Fiji **Log** so you can check that every track lands on a real TIFF frame. When the box says all tracks map correctly, confirm.

Why it matters: a wrong offset silently pairs each track point with the wrong image, giving wrong depths. Take a moment to confirm the verdict looks good.

5 Choose how to sample depth

Pick how the plugin reads pixels under each detection:

- **Sampling method:** *Radius* (average a small disk — good default), *4-Neighbor*, or *Single Pixel*. You can also choose **All** to run every method.
- **Aggregation:** *Median* (robust to odd pixels — recommended) or *Mean*. (Disabled automatically for Single Pixel, since there's only one value.)
- **Pixel convention:** leave on *Corner* (the default) unless you have a specific reason to use *Center*.

Not sure? Radius + Median + Corner is a solid default for most users.

6 Choose output folder & formats

Pick where to save results and tick the formats you want (see the next section). Click OK to run.

What you get out

Format	What it's for
.npy files (<code>tracks_2D/</code> , <code>tracks_3D/</code>)	For Python analysis/plotting. 3D files hold [X, Y, Z, T].
Results Table CSV	Open in Fiji via <code>Analyze > Import > Results...</code> , or in Excel.
ROI set (.zip)	Open in Fiji's ROI Manager (<code>More >> Open...</code>) to overlay points on your image. Optional XZ/YZ side-view sets are also available.
export_report.txt	A plain-text summary of every track — how many points were kept or dropped, and why. Check this if numbers look off.

X and Y are in **pixels**, Z is in **micrometers**, and T is the frame number.

Good to know

- **No track is thrown away.** If one point is bad (e.g. its frame is missing or it falls outside the image), only that single point is dropped — the rest of the track is still exported.
- **Dropped points leave a gap** in the frame numbers rather than renumbering — this is intentional.
- **Check the Log window** after running. It reports how many points were extracted and how many were skipped, and why. Its summary line counts the dropped points by reason: `invalidXY` (the CSV's X or Y was not a number), `missingFrame` (no image for that frame), `outOfBounds` (the point lies outside the image), `unmappedIndex` (the pixel's code is not in the depth map) and `noData` (TopoJ images only — no depth could be read there, either because TopoJ found no surface or because the value was ambiguous; see Part 2).
- If you picked **All** in the method step (**Step 5** standard, **Step 6** TopoJ), each method gets its own sub-folder inside your output folder.

Part 2 (alternative) — 3D Z-Extractor (TopoJ / direct-Z)

Use this instead of the standard extractor when your depth images come from **Fiji's TopoJ**, with no `.json` map alongside them.

Read this before anything else: a TopoJ image does not contain depths. TopoJ writes a *counter*. Each pixel holds `(slice count - k) × voxel depth` , where `k` is the slice of your **source** stack that was brightest at that pixel, and *voxel depth* is whatever Fiji had recorded for that stack **at the moment TopoJ ran** — which is **1.0 if nobody calibrated it**. Nothing in the file records the real depths, so the plugin has to be told how to convert them. That is what **Step 2** is for, and getting it wrong silently changes every depth in your results.

Open it via **Plugins > ZTracker > 3D Z-Extractor (TopoJ / direct-Z)**.

Step 2 — Z calibration. Four numbers, all read from the **source stack you ran TopoJ on** (`Image > Properties...`). None of them is stored in the TopoJ output.

- **Z of first slice (k = 0), μm** — the physical depth of that stack's first slice.

- **Z step per slice, μm** — the **true** physical spacing between its slices.
- **Source stack slice count** — how many slices it had.
- **Encoding scale (voxel depth TopoJ used)** — the voxel depth Fiji held **when TopoJ ran. It is 1.0 if nobody calibrated the stack**, even when the true step is 2 μm or anything else.

The last two bullets are the ones to be careful with: **Z step and encoding scale are different quantities**. They are equal only if the stack was calibrated correctly before TopoJ ran. If TopoJ ran uncalibrated and you enter your true step as the encoding scale, every depth in the run comes out rescaled — with no error and nothing to notice.

If you see a WARNING that the value 1.0 is ambiguous, read it. TopoJ writes a literal `1.0` into any pixel it never assigned — meaning the surface was at or above the very top of the stack — and does *not* scale that literal by the voxel depth. So when your encoding scale is 1.0 (or 0.5, 0.25, 0.2...), `1.0` is **also** a legitimate depth, the one for the deepest slice, and nothing in the file says which a given pixel means. The plugin refuses to guess: those pixels are dropped as **no data**, detections landing on them report no depth, and the depth of that one slice cannot appear anywhere in the run — so your Z range looks short at one end for a reason you would otherwise never see. The warning states both depths the value could have meant and how many pixels were dropped. **The fix is to set the source stack's voxel depth in Fiji (Image > Properties > Voxel depth) before running TopoJ, then re-run TopoJ and enter that value as the encoding scale.** At a voxel depth of 2, for instance, `1.0` is no longer a legitimate value and the ambiguity disappears entirely.

If the numbers do not match the images, the run stops with a message naming how many pixels could not be decoded, up to five of them with their frame and position, and what they were checked against. Nothing is converted in that case — you never get a half-decoded result.

What else is different from the standard extractor:

- **No Z-mapping JSON.** Step 1 asks for just **two** inputs — the **TopoJ Z-map TIFF folder** and your **tracking CSV**.
- **32-bit float images only.** (16-bit/8-bit images are rejected with a clear message — those belong to the standard, indexed extractor.)
- **Filenames must end with the frame number.** The frame index is read from the number your filename **ends with**, so any name prefix works and the zero-padding can be any width — `frame7.tif`, `topoj_0007.tif`, and `height_map_00000100.tif` all load fine (as frames 7, 7, and 100). A file that does **not** end with a number (e.g. `topoj_0007_final.tif`) is **rejected with a message listing the offending names** — rename it so the frame number comes last, right before `.tif`.
- **Seven steps, not six.** Step 2 is the extra one, so every later step is numbered one higher than in the standard extractor: CSV format is Step 3, columns Step 4, the frame-offset check Step 5, the method choices Step 6, and output Step 7.

Everything else is identical to the standard extractor: the same CSV format and column steps, the same live frame-offset check, the same sampling/aggregation/pixel-convention choices, the same output folders and formats, and the same per-point drop behaviour and reports. X/Y stay in pixels, Z is in μm , T is the frame number.

Not sure which one you have? If your projection folder came with a `z_layer_mapping.json`, use the

standard extractor. If it's a bare folder of 32-bit float TIFFs from TopoJ with no JSON, use this **direct-Z** one.

Quick troubleshooting

Symptom	Likely fix
Lots of points show "missing frame"	Revisit the frame-offset step (Step 4 in the standard extractor, Step 5 in TopoJ) — the offset is probably wrong (try +1 or 0).
Lots of points show "out of bounds"	Check your CSV X/Y match the TIFF image size, and check the pixel convention in the method step (Step 5 standard, Step 6 TopoJ).
Plugin not in the menu	Restart Fiji, or use <code>Help > Refresh Menus</code> .
Nothing seems to export	Make sure at least one format is ticked in the output step (Step 6 standard, Step 7 TopoJ), and read <code>export_report.txt</code> .
"Inconsistent image dimensions in TIFF folder"	One TIFF is a different size from the first one in the folder — the message names it and gives both sizes. Remove it or re-crop everything to one size. Part 1 keeps this from happening within a single run over one dataset, but combining two runs' output into one folder — or assembling a folder by hand — still can.
"requires 32-bit float TIFFs" error	You opened the TopoJ / direct-Z extractor with indexed images — use the standard 3D Z-Coordinate Extractor (with its JSON map) instead.
Part 1: the run finds no data, or fewer datasets than you expected	Check the Input type matches your folder's layout first, then the Scope , then the input folder itself — the grey structure line under Scope spells out what is expected. In Batch, a dataset whose depth folders are all misnamed is skipped without a message, so a lower-than-expected dataset count is worth chasing.
Part 1: finished, but the Log ends with a WARNING about skipped timepoints	Those time frames are missing from the output — scroll up to the <code>skipped timepoint</code> lines for the reason on each. Don't treat the run as complete: Part 2 will read the result as a normal gap in the frame numbering and say nothing.
Part 1: "slice has no readable Z in its label"	Your TIFF stack's slices aren't labelled with their depth (<code>z = -400.000</code>). Re-save the stack from Fiji with the depth labels, or use the Z-layer sub-folder layout instead.